

Image Recognition



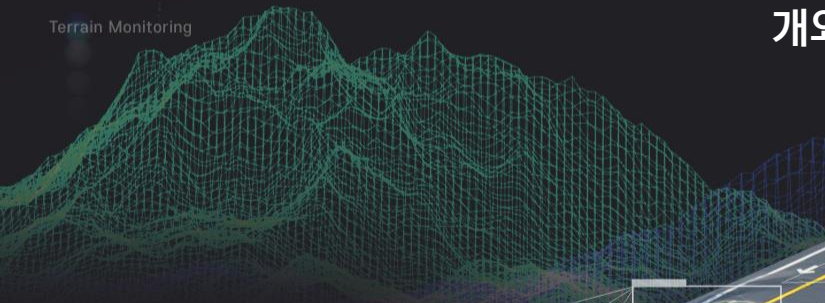
강원혁신플랫폼

컴퓨터 비전



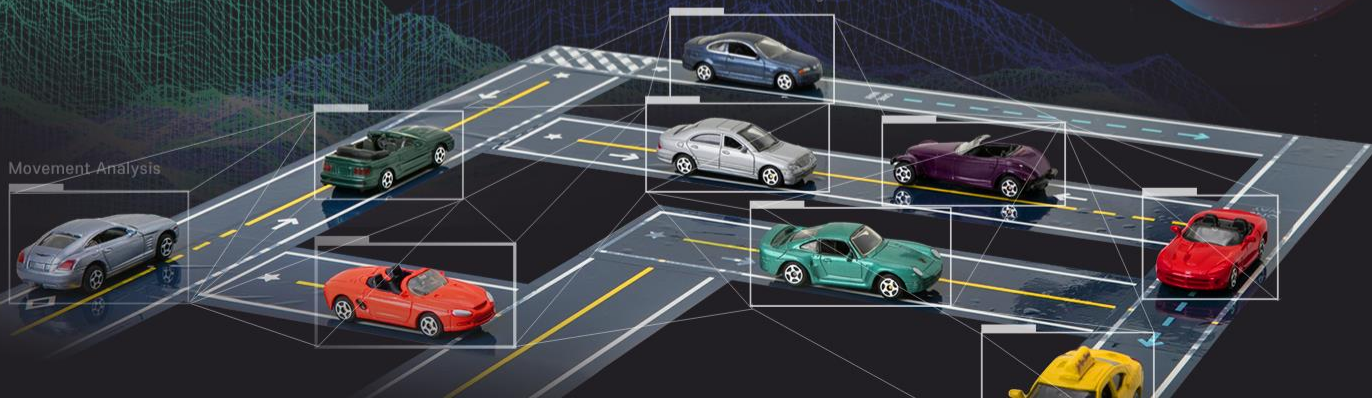
개와 고양이 이진 분류

Terrain Monitoring



Autonomous Driving

Movement Analysis



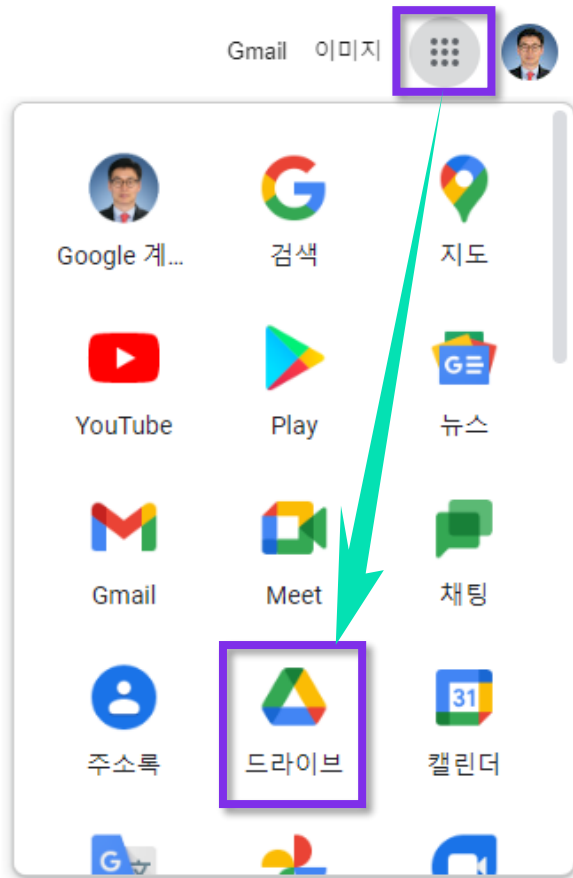
개와 고양이 분류(Cats vs Dogs Classification)

고양이 이미지 1,000장과 개 이미지 1,000장, 총 2,000장의 이미지를 학습시켜서 새로운 개나 고양이 이미지를 입력했을 때 그것이 개인지 고양이인지를 분류해 내는 모델을 만들어 보자.

이진 이미지 분류(Image Binary Classification)

- ② 인공지능(딥러닝)의 세부 알고리즘은 차차 배우기로 하고, 개와 고양이 분류의 전체과정과 실습을 통하여 학습해 보자.
- ② 실습은 구글 Colab을 이용한다.

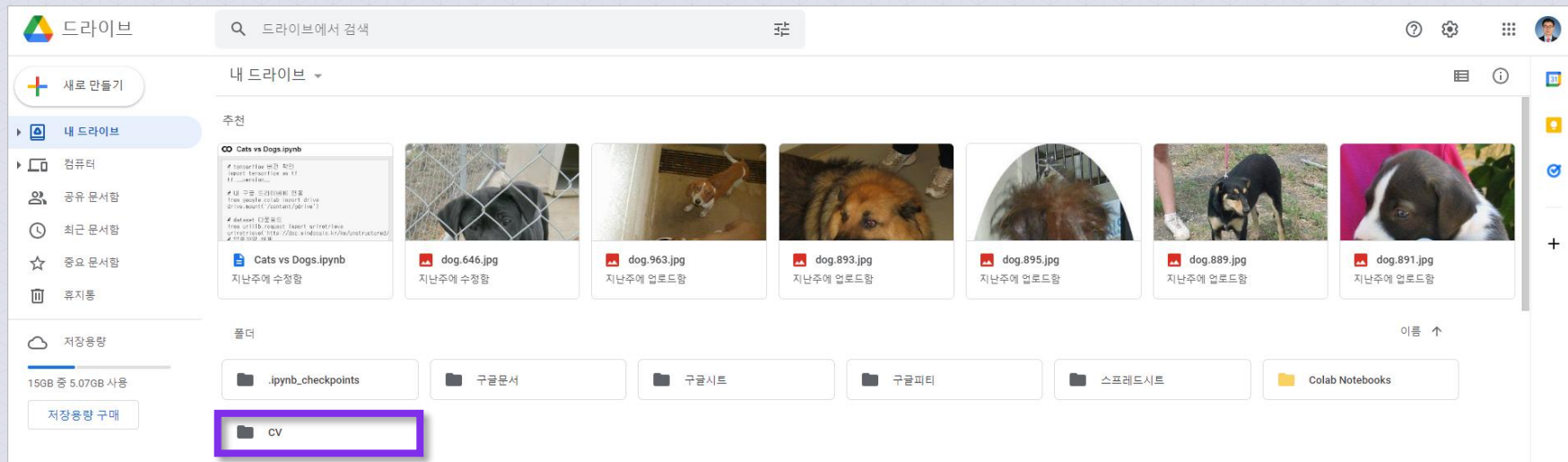
① 실습 파일 복사하기



- 구글 Colab을 이용하기 위해
구글 크롬 브라우저에서 그림과 같이
구글 드라이브를 클릭한다.

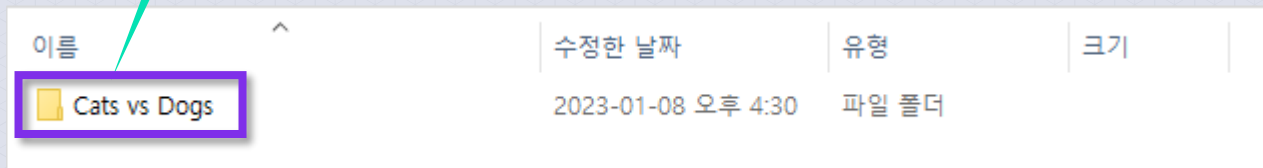
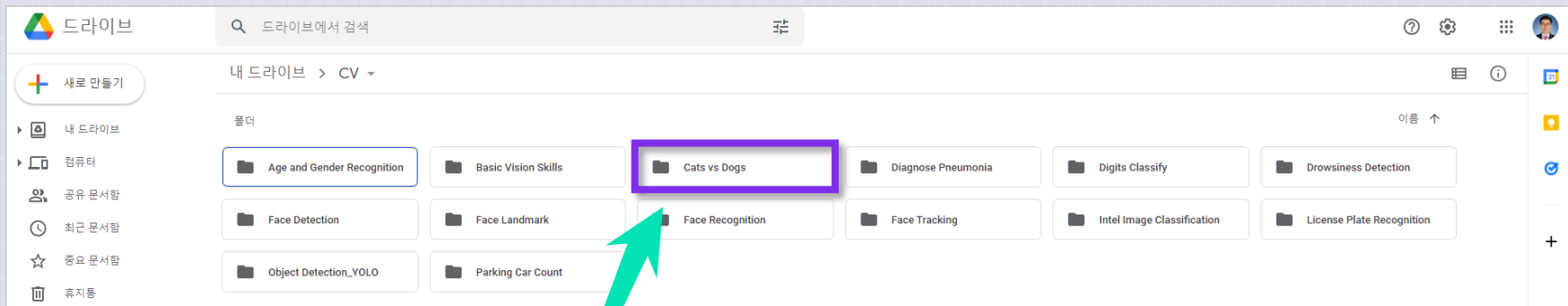
① 실습 파일 복사하기

구글 드라이브에서 CV 폴더를 클릭한다.

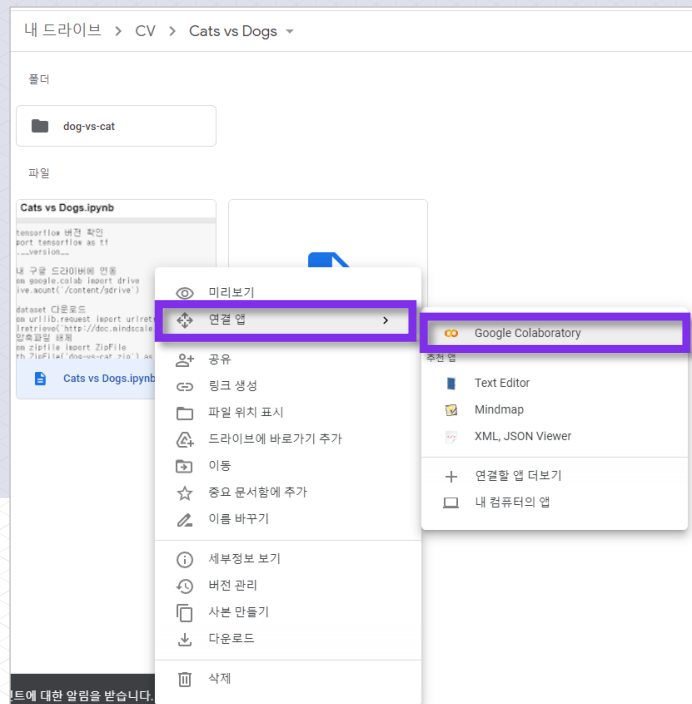


① 실습 파일 복사하기

- 아래와 같이 실습 폴더 “Cats vs Dogs”를 구글 드라이브에 복사한다.
- 구글 드라이브 > CV 폴더 > Cats vs Dogs 폴더를 클릭한다.



- 아래의 그림의 Cats vs Dogs 폴더에서 **Cats vs Dogs.ipynb**를 선택 후 오른쪽 마우스를 클릭하여 연결 앱 > Google Colaboratory를 클릭한다.



① 실습 파일 복사하기

아래 그림과 같이 구글 Colab 노트북 화면이 보이면 실습 준비가 완료된 것이다.



```
Cats vs Dogs.ipynb ☆
파일 수정 보기 삽입 런타임 도구 도움말 8월 21일에 마지막으로 저장됨

+ 코드 + 텍스트

# tensorflow 버전 확인
import tensorflow as tf
tf.__version__

'2.8.2'

[ ] # 내 구글 드라이브에 연동
from google.colab import drive
drive.mount('/content/gdrive')

Drive already mounted at /content/gdrive; to attempt to forcibly remount, call drive.mount("/content/gdrive", force_remount=True).

[ ] # dataset 다운로드
from urllib.request import urlopen
urlopen('http://doc.mindscale.kr/km/unstructured/dog-vs-cat.zip', 'dog-vs-cat.zip')

# 압축파일 해제
from zipfile import ZipFile

with ZipFile('dog-vs-cat.zip') as z:
    z.extractall()

[ ] # dataset을 내 구글 드라이브로 이동시킴
!mv dog-vs-cat /content/gdrive/My Drive/CV/Cats vs Dogs/
```


- 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



```
# tensorflow 버전 확인  
import tensorflow as tf  
tf.__version__
```

```
'2.8.2'
```

실행 결과 tensorflow 버전이 '2.8.2'인 것을 알 수 있다.

③ 구글 드라이브에 연동하기

🔄 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



```
# 내 구글 드라이브에 연동
```

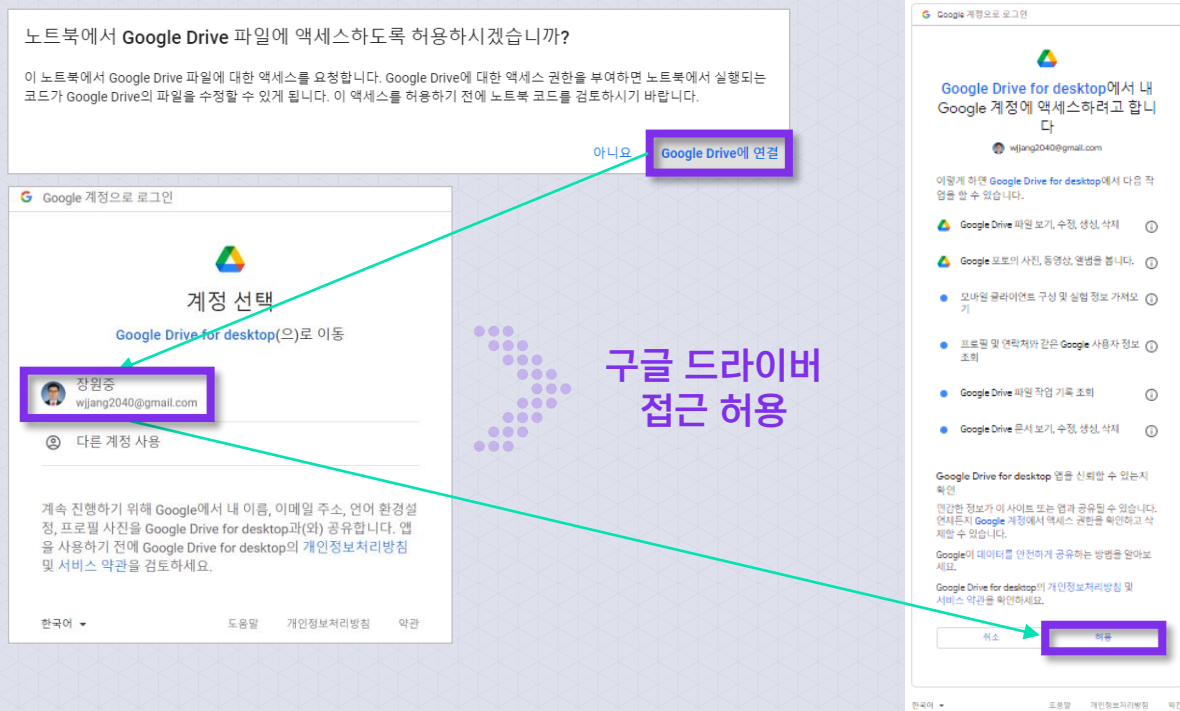
```
from google.colab import drive
```

```
drive.mount('/content/gdrive')
```

```
Drive already mounted at /content/gdrive; to attempt to forcibly remount, call drive.mount("/content/gdrive", force_remount=True).
```

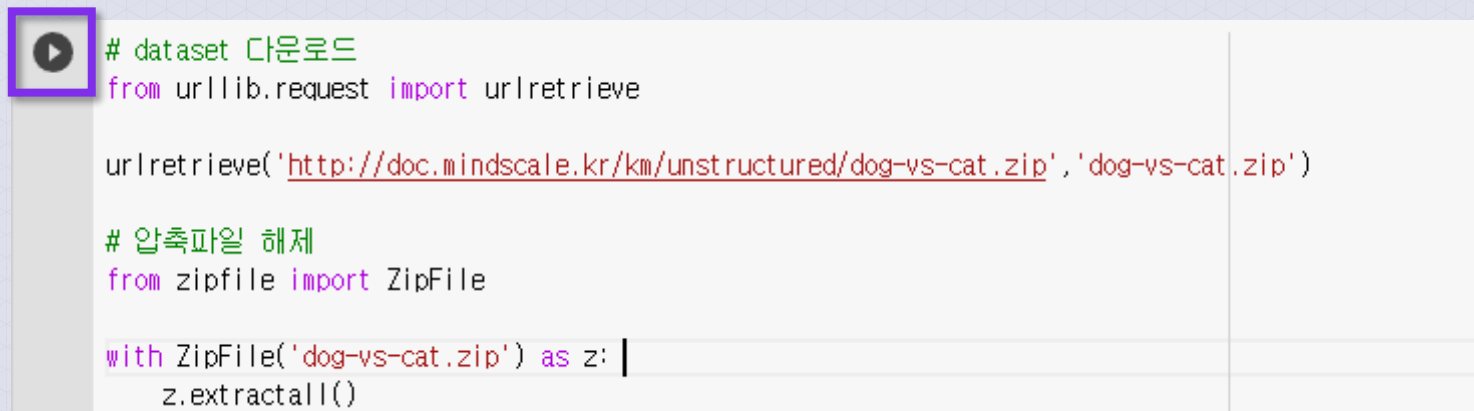
③ 구글 드라이브에 연동하기

- ① [Google Drive에 연결] > [계정 선택] > [구글 드라이브 접근 허용] 메뉴를 선택한다.
- ② gdrive에 연동되면 구글 드라이브에 저장된 데이터를 활용할 수 있다.



④ 훈련에 사용할 데이터 셋을 dog-vs-cat.zip 파일 이름으로 VM에 다운받기

- 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



```
# dataset 다운로드
from urllib.request import urlretrieve

urlretrieve('http://doc.mindscale.kr/km/unstructured/dog-vs-cat.zip', 'dog-vs-cat.zip')

# 압축파일 해제
from zipfile import ZipFile

with ZipFile('dog-vs-cat.zip') as z: |
    z.extractall()
```

- 실행 결과 VM에 dog-vs-cat.zip 파일을 다운로드하여 doc-vs-cat 폴더에 압축이 해제된 것을 볼 수 있다.
- [doc-vs-cat] > [train] > [cat]
> [dog]
> [validation] > [cat]
> [dog]

- 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



필요한 패키지와 모듈을 불러옴

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, Dense, Flatten, Dropout, MaxPooling2D
from tensorflow.keras.optimizers import RMSprop
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping
```

- 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



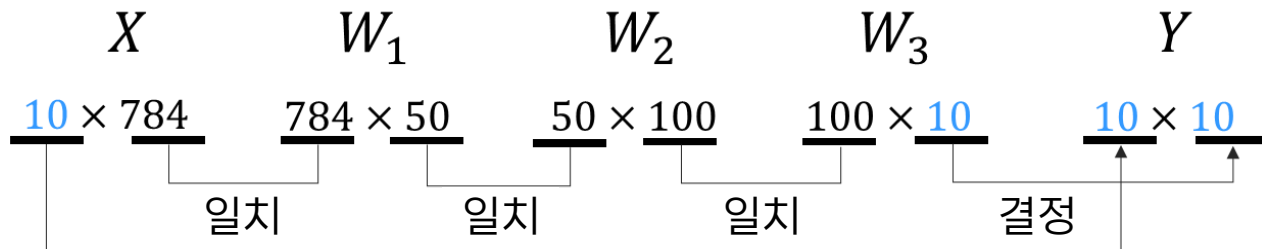
```
image_height = 150 # 이미지 높이  
image_width = 150 # 이미지 넓이  
batch_size = 10 # batch size  
no_of_epochs = 50 # training 횟수
```

- 위 코드에서 학습을 위한 배치 크기=batch_size=10, 학습 횟수(no_of_epochs)=50으로 설정한 것을 볼 수 있다.
- 배치 크기 = 10 : 10장 단위로 묶어서 학습



배치 처리

- 이미지 10장을 하나의 입력 데이터로 표현하여, 신경망 각 층의 배치 처리를 위한 배열 형상 추이를 정리해 보면 다음과 같음



- 위 형상에서 입력 데이터의 형상은 10×784 , 출력 데이터의 형상은 10×10 이 됨
- 은닉층(첫 번째 50개 뉴런, 두 번째 100개 뉴런, 세 번째 10개 뉴런)으로 구성됨
- 10장 분량의 입력 데이터의 결과가 한 번에 출력됨을 나타냄
- 이처럼 하나로 묶은 입력 데이터를 배치(Batch)라고 함

배치 처리



■ 이미지 1장당 처리 시간을 대폭 줄여주며, 컴퓨터로 계산할 때 큰 이점을 줌

- 1 수치 계산 라이브러리 대부분이 큰 배열을 효율적으로 처리할 수 있도록 고도로 최적화되어 있기 때문임
 - ② 커다란 신경망에서는 데이터 전송이 병목으로 작용하는 경우가 자주 있는데, 배치 처리를 함으로써 버스에 주는 부하를 줄임
- 2 느린 I/O를 통해 데이터를 읽는 횟수가 줄어, 빠른 CPU나 GPU로 순수 계산을 수행하는 비율이 높아짐
 - ② 즉, 배치 처리를 수행함으로써 큰 배열로 이뤄진 계산을 하게 됨
 - ② 컴퓨터에서는 큰 배열을 한꺼번에 계산하는 것이 분할된 작은 배열을 여러 번 계산하는 것보다 빠름

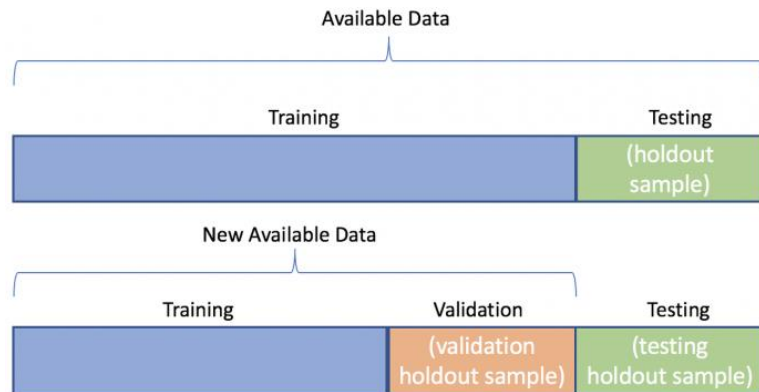
훈련 데이터와 시험 데이터

■ 기계학습 문제는 데이터를 **훈련 데이터(Training data)**와 **시험 데이터(Test data)**로 나눠 학습과 실험을 수행하는 것이 일반적임

- ➡ 우선 훈련 데이터만 사용하여 학습하면서 최적의 매개변수를 찾음
- ➡ 그런 다음 시험 데이터를 사용하여 앞서 훈련한 모델의 실력을 평가함

■ 왜 훈련 데이터와 시험 데이터를 나눠야 할까?

- ➡ 우리가 원하는 것은 범용적으로 사용할 수 있는 모델이기 때문임
- ➡ 범용 능력을 제대로 평가하기 위해 훈련 데이터와 시험 데이터를 분리하는 것임



훈련 데이터와 시험 데이터

■ 범용 능력은 아직 보지 못한 데이터(훈련 데이터에 포함되지 않는 데이터)로도 문제를 올바르게 풀어내는 능력임

↻ 범용 능력을 획득하는 것이 기계학습의 최종 목표

예 손글씨 숫자 인식의 최종 결과

특정인의 특정 글자 X

임의의 사람의 임의의 글자 O

- 만약 수중에 있는 훈련 데이터만 잘 판단한다면 그 데이터에 포함된 사람의 글씨체만 학습했을 가능성이 큼
- 그래서 데이터셋 하나로만 매개변수의 학습과 평가를 수행하면 올바른 평가가 될 수 없음

오버피팅(Overfitting)

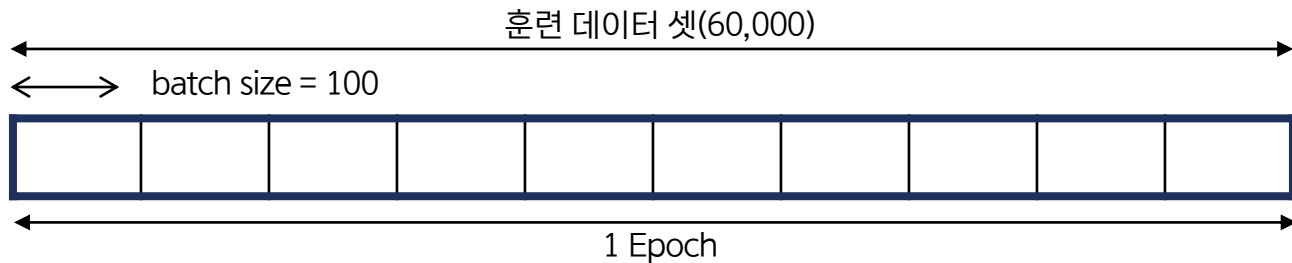
하나의 데이터 셋에만 지나치게 최적화된 상태

미니배치 학습



■ 60,000장의 훈련 데이터 중에서 100장을 무작위로 뽑아, 100장만을 사용하여 학습

- ➡ MNIST 데이터셋에서 훈련 데이터 60,000개를 100개의 미니배치로 학습할 경우, 600회 반복하면 모든 훈련 데이터를 소진하게 됨
- ➡ 이 경우 600회가 1 에폭(epoch)이 됨



- ➡ 미니배치(minibatch) : 데이터 셋을 batch size(위 예의 경우 100개) 크기로 쪼개서 학습
- ➡ 1 epoch(모든 데이터 셋을 한번 학습) = $60,000 / \text{batch size}(=100) = 600 \text{ iteration}$
- ➡ 1 iteration(1회 학습) = 100개 데이터에 대해서 학습

ImageDataGenerator 클래스를 통해 `train_datagen` 인스턴스를 만든다.

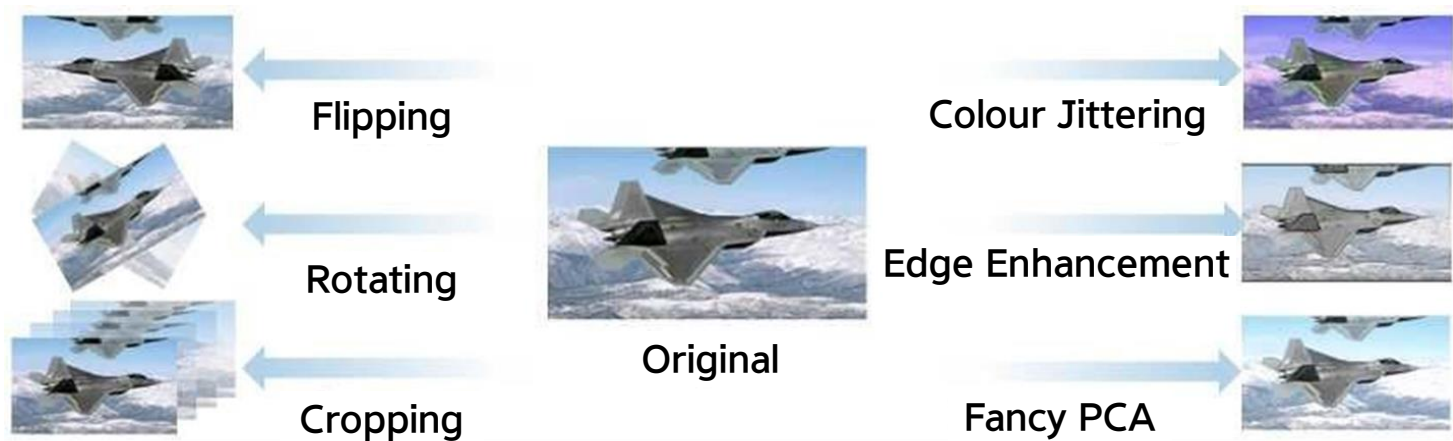
🔄 구글 Colab 노트북 화면에서 **실행 버튼**을 클릭하면 됨



```
# ImageDataGenerator 클래스를 통해 train_datagen 인스턴스를 만듭니다.  
train_datagen = ImageDataGenerator(  
    rescale=1./255,          # 픽셀 값을 0~1 범위로 변환  
    rotation_range=40,       # 40도까지 회전  
    width_shift_range=0.2,   # 20%까지 좌우 이동  
    height_shift_range=0.2,  # 20%까지 상하 이동  
    shear_range=0.2,        # 20%까지 기울임  
    zoom_range=0.2,         # 20%까지 확대  
    horizontal_flip=True,    # 좌우 뒤집기  
)
```

ImageDataGenerator 클래스를 통해 `train_datagen` 인스턴스를 만든다.

🔄 tensorflow.keras의 ImageDataGenerator 클래스는 데이터 증강(Data Augmentation)을 지원



여러 가지 데이터 증강 기법

데이터 증강(Data Augmentation)



■ 이미지 회전과 무작위 변환을 적용해 훈련 데이터의 다양성을 증가시키는 기술

- ➡ 데이터의 양을 늘려 과적합 방지
- ➡ 훈련 데이터에만 적용
- ➡ 다양한 데이터 변형 상황을 대비

예] 숫자 6, 9 이미지와 같은 데이터일 경우 오히려 학습에 방해가 될 수 있으므로 데이터의 정보를 파악하고 적당히 변형시켜야 함(회전 10~20% 정도)

이미지 증강

예시

크기 조절, 회전, 밝기, 색깔 변경, 잘라내기

데이터 증강(Data Augmentation)



■ 데이터 증강(Data Augmentation)은 왜 필요할까?

1 ▶ 과적합(오버피팅) 문제

- ⊕ 정형 데이터를 예측하는 일반적인 머신러닝 모델과 마찬가지로, 이미지를 분류하는 CNN 모델도 극복해야 할 문제
- ⊕ CNN 모델의 성능을 높이면서 오버피팅을 극복할 수 있는 가장 좋고 근본적인 해결책은 학습 데이터의 다양성을 늘리는 것임

데이터 증강(Data Augmentation)

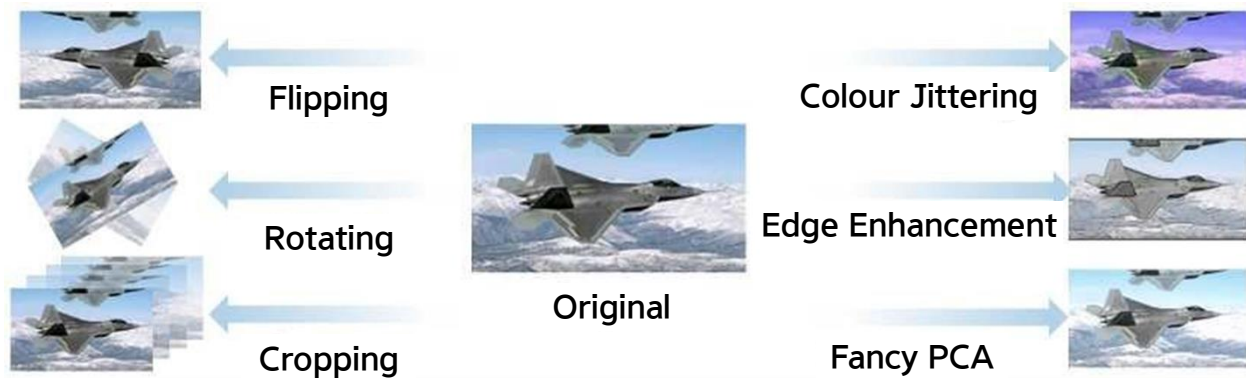


■ 데이터 증강(Data Augmentation)은 왜 필요할까?

2 이미지 데이터 추가

- ❖ 학습 데이터의 개수를 직접 늘리면 더할 나위 없이 좋지만 이미지 데이터를 추가로 구하는 것은 사람의 손을 필히 거쳐야 하며 비용도 시간도 많이 드는 일임
- ❖ 비용과 시간도 적게 소요하면서 학습 데이터의 다양성을 늘릴 수 있는 방법으로 고안된 것이 **데이터 증강 기법**
 - └ 하나의 원본 이미지를 다양한 버전으로 만들어 학습시키는 것

데이터 증강(Data Augmentation)



여러 가지 데이터 증강 기법

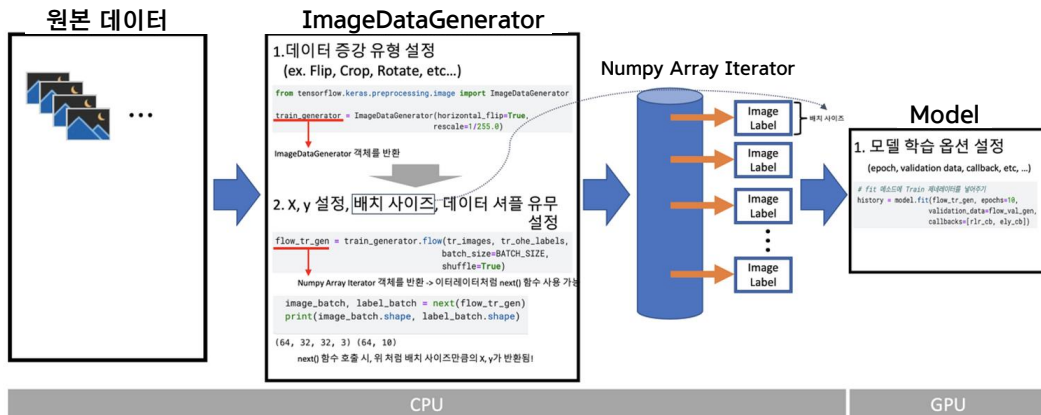
- Flipping : 원본 이미지를 수평 또는 수직으로 반전시키는 방법
- Rotating : 회전시키는 방법
- Cropping : 일부를 자르는 방법
- Colour Jittering : RGB 채널 순서를 바꾸거나 픽셀값을 변경하여 원본 이미지의 색조를 변경시키는 방법

데이터 증강(Data Augmentation)

텐스플로에서는 원본 이미지 증강 기법을 tensorflow.keras의 **ImageDataGenerator** 클래스가 대신함

ImageDataGenerator 클래스의 역할

- 원본 이미지 데이터를 증강
- 텐스플로 내에서 하나의 데이터 파이프라인을 구축
- Tensorflow.keras의 preprocessing 모듈에 존재



ImageDataGenerator 클래스의 과정

※ ImageDataGeneratorExam.ipynb
프로그램 실습을 통해
좀 더 깊이 있게 학습해 보자.

좀 더 구체적으로 ImageDataGenerator 과정을 정리해 보면 다음과 같다.

1 ImageDataGenerator 객체를 생성하면서 데이터 증강을 수행할 유형들을 지정함

```

1  # ImageDataGenerator 클래스를 통해 train_datagen 인스턴스를 만듭니다.
2  train_datagen = ImageDataGenerator(
3      rescale=1./255,          # 픽셀 값을 0~1 범위로 변환
4      rotation_range=40,       # 40도까지 회전
5      width_shift_range=0.2,   # 20%까지 좌우 이동
6      height_shift_range=0.2, # 20%까지 상하 이동
7      shear_range=0.2,        # 20%까지 기울임
8      zoom_range=0.2,         # 20%까지 확대
9      horizontal_flip=True,    # 좌우 뒤집기
10 )
    
```

☞ 위 코드와 같이 다양한 데이터 증강 기법을 설정하는 것을 볼 수 있음

좀 더 구체적으로 ImageDataGenerator 과정을 정리해 보면 다음과 같다.

2 1단계에서 생성한 ImageDataGenerator 객체를 생성한 후 훈련 데이터에서 배치 크기 만큼 이미지를 만들어 냄

- ➡ 학습 이미지(X), 정답 레이블(y) 이미지의 배치 사이즈 설정
- ➡ 데이터 셔플 여부 지정
- ➡ 반환되는 레이블 배열의 유형 지정
- ➡ Numpy Array Iterator(iterator 안의 데이터를 하나씩 호출)

```
1 # train_datagen 인스턴스를 통해 train dataset 에서 한번에 batch_size 만큼 이미지를 만들어내는 train_generator 객체를 생성한다.
2
3 # flow_from_directory() : directory 경로로부터 데이터프레임을 가져오고 augmented 또는 normalized된 데이터 배치를 생성한다.
4 # flow_from_directory()는 이름에서도 알 수 있듯이 어떤 디렉토리와 관련된 함수라는 것을 알 수 있다.
5 # - 여기서 어떤 디렉토리란 무엇일까? 바로 메타 데이터가 담긴 디렉토리를 의미한다.
6
7 train_generator = train_datagen.flow_from_directory('gdrive/My Drive/CV/Cats vs Dogs/dog-vs-cat/train',
8           batch_size=10,          # Size of the batches of data (default: 32).
9           class_mode='binary', # Determines the type of label arrays that are returned. |
10           # "categorical" will be 2D one-hot encoded labels, "binary" will be 1D binary labels, "sparse" will be b
11           target_size=(image_width, image_height)) # 이미지의 크기 재정의
```

- ➡ 위와 같이 설정하는 이유는 이미지 사이즈가 커질수록 한번에 이미지를 CPU, GPU에 올릴 수 없기 때문에 배치 사이즈 크기 설정 같은 처리를 미리 수행함

좀 더 구체적으로 `ImageDataGenerator` 과정을 정리해 보면 다음과 같다.

3 모델을 학습시키기 위해 필요한 하이퍼파라미터를 설정함

- 2단계에서 만들어진 `iterator`를 생성된 모델의 메서드(`fit` 또는 `fit_generator`)에 입력
- 에폭(반복 학습 횟수) 설정
- 콜백함수(모델이 더 이상 학습을 못할 경우 종료) 설정

```

1  # model training(모델을 학습시킵니다)
2  history = model.fit_generator(generator=train_generator,
3                               steps_per_epoch=2000//batch_size,
4                               epochs=no_of_epochs,
5                               validation_data=test_generator,
6                               validation_steps=800//batch_size,
7                               callbacks=[monitor_val_acc])
8  # 정확도를 모니터링하다가 증가되지 않고 떨어지거나 변화가 거의 없다면 patience 만큼 기다리다 학습을 중단함
    
```

`train_datagen` 인스턴스를 통해 훈련 데이터 셋에서 한번에 `batch_size` 만큼 이미지를 만들어내는 `train_generator` 객체를 생성한다.

train_datagen 인스턴스는 ImageDataGenerator 클래스의 객체

```
# train_datagen 인스턴스를 통해 train dataset 에서 한번에 batch_size 만큼 이미지를 만들어내는 train_generator 객체를 생성한다.

train_generator = train_datagen.flow_from_directory('gdrive/My Drive/CV/Cats vs Dogs/dog-vs-cat/train',
                                                    batch_size=10, # Size of the batches of data (default: 32).
                                                    class_mode='binary', # Determines the type of label arrays that are returned.
                                                    # "categorical" will be 2D one-hot encoded labels, "binary" will be 1D binary labels, "sparse" will be 1D integer labels
                                                    target_size=(image_width, image_height)) # 이미지의 크기 재정의
```

Found 2000 images belonging to 2 classes.

- flow_from_directory() 함수는 **directory** 경로로부터 데이터프레임을 가져오고, **증강(Augmented)** 또는 **정규화(Normalized)**된 **데이터 배치**를 생성함
- 위 코드에서 훈련용 데이터 셋에서 **batch_size=10** 만큼 **이미지**로 **train_generator** 객체가 만들어진 것을 알 수 있음
- 실행 결과 **훈련 데이터 셋**으로 **2개의 클래스**로 구성된 **2,000장의 이미지**가 있음을 알 수 있음

ImageDataGenerator 클래스를 통해 검증용 데이터 셋(Validation data set)에서 한번에 batch_size 만큼 이미지를 만들어내는 test_generator 객체를 생성한다.

```
# ImageDataGenerator를 통해 validation dataset 에서 한번에 batch_size 만큼 이미지를 만들어내는 test_generator 객체를 생성한다.  
test_generator = ImageDataGenerator(rescale=1./255).flow_from_directory(  
    'gdrive/My Drive/CV/Cats vs Dogs/dog-vs-cat/validation',  
    batch_size=10,  
    class_mode='binary',  
    target_size=(image_width, image_height))
```

Found 800 images belonging to 2 classes.

- 위 코드에서 검증용 데이터 셋에서 batch_size=10 만큼 이미지로 test_generator 객체가 만들어진 것을 알 수 있음
- 실행 결과 검증용 데이터 셋으로 2개의 클래스로 구성된 800장의 이미지가 있음을 알 수 있음

ImageDataGenerator 클래스를 통해 검증용 데이터 셋(Validation data set)에서 한번에 batch_size 만큼 이미지를 만들어내는 pred_generator 객체를 생성한다.

```
# ImageDataGenerator를 통해 validation dataset 에서 한번에 batch_size 만큼 이미지를 만들어내는 pred_generator 객체를 생성한다.  
pred_generator = ImageDataGenerator(rescale=1./255).flow_from_directory(  
    'gdrive/My Drive/CV/Cats vs Dogs/dog-vs-cat/validation',  
    batch_size=1,  
    class_mode='binary',  
    target_size=(image_width, image_height))
```

Found 800 images belonging to 2 classes.

- 위 코드에서 검증용 데이터 셋에서 batch_size=1 만큼 이미지로 pred_generator 객체가 만들어진 것을 알 수 있음
- 실행 결과 검증용 데이터 셋으로 2개의 클래스로 구성된 800장의 이미지가 있음을 알 수 있음

`train_generator` 객체가 만들어 내는 이미지의 클래스 인덱스를 확인해 보자.

```
# train_generator 가 만들어 내는 이미지의 클래스 인덱스가 무엇인지 확인해 본다.  
|  
print(train_generator.class_indices)  
  
{'cat': 0, 'dog': 1}
```

- ➡ 실행 결과 고양이는 0, 개는 1임을 알 수 있음
- ➡ 위 코드에서 `class_indices` 객체 변수의 자료구조는 딕셔너리인 것을 알 수 있음

다음은 레이블 명(label name)을 찾아오는 findKey() 함수를 정의한다.

```
# label name 을 찾아 오는 함수를 정의한다.  
def findKey(indices, search_value):  
    for key, value in indices.items():  
        if(value == search_value):  
            return key  
    return -1
```

- 위 코드에서 findKey() 함수는 indices 객체(딕셔너리)의 value와 search_value의 값이 동일한 경우, indices 객체의 key 값을 반환함

예 indices = {'cat':0, 'dog':1}, search_value = 0인 경우, findKey() 함수의 반환값은 'cat'이 됨



다음은 훈련용 데이터 셋에서 샘플 데이터(16장의 이미지)를 출력해 보자.



```
# train dataset 에서 샘플 데이터를 display 해본다.
for X_batch, y_batch in ImageDataGenerator(rescale=1./255).flow_from_directory('gdrive/My Drive/CV/Cats vs Dogs/dog-vs-cat/train',
                                          batch_size=16, class_mode='binary', target_size=(150, 150)):

    plt.figure(figsize=(20,20))
    # create a grid of 4x4 images
    for i in range(0, 16):
        ax = plt.subplot(4, 4, i+1)
        ax.set_title(findKey(train_generator.class_indices, y_batch[i])) # 이미지의 label name 을 찾아온다
        plt.imshow((X_batch[i].reshape(150, 150, 3)*255).astype(np.uint8)) # train 샘플 이미지를 보여준다.
    # show the plot
    plt.show()
    break
```

위 코드에서 16장의 이미지를 출력하고 반복문을 종료한 것을 알 수 있음

- 위 코드의 첫 번째 반복문은 훈련용 데이터를 16장 단위로 묶어서 읽어옴
- 두 번째 반복문은 16번 반복하면서 첫 번째 반복문에서 읽어온 16장 이미지 각각의 레이블을 이미지 제목으로 (150, 150, 3) 형상으로 화면에 출력함
- 두 번째 반복문이 끝나면 16장의 이미지를 화면에 출력하고 종료함

개와 고양이 분류(학습)

다음은 훈련용 데이터 셋에서 샘플 데이터(16장의 이미지)를 출력해 보자.

실행 결과 16장의 이미지는 아래 그림과 같음



다음은 인공지능망 네트워크를 구성하여 이미지 분류를 위한 모델을 만든다.

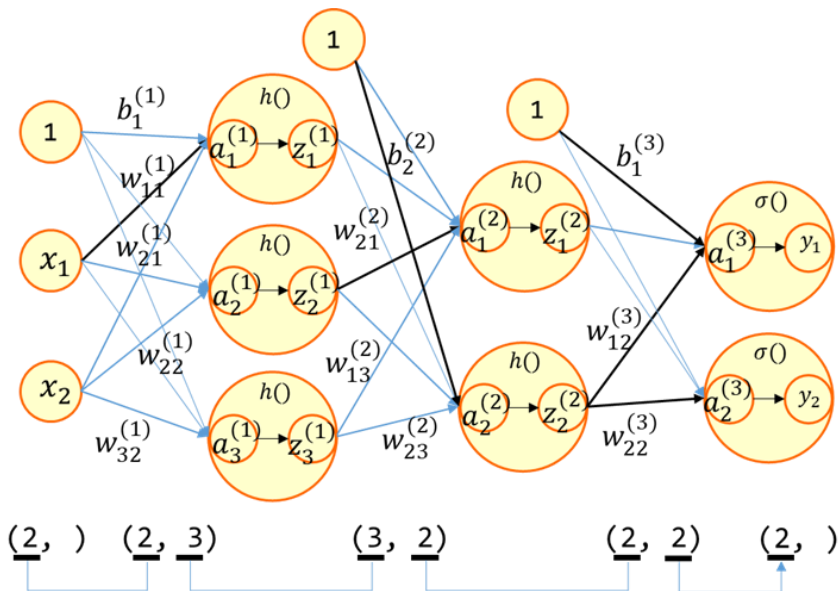
- ⊕ CNN, Fully connected neural network, Relu, Sigmoid, MaxPooling, Cross entropy error, RMSprop, Accuracy, Dropout 알고리즘을 이용함

```
# create model(모델을 만듭니다)
model = Sequential()
model.add(Conv2D(32, (3, 3), activation='relu', input_shape=(image_height, image_width, 3)))
model.add(MaxPooling2D((2, 2)))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D((2, 2)))
model.add(Conv2D(128, (3, 3), activation='relu'))
model.add(MaxPooling2D((2, 2)))
model.add(Conv2D(128, (3, 3), activation='relu'))
model.add(MaxPooling2D((2, 2)))
model.add(Flatten())
model.add(Dropout(0.5))
model.add(Dense(512, activation='relu'))
model.add(Dense(1, activation='sigmoid'))
model.compile(loss='binary_crossentropy', optimizer=RMSprop(lr=1e-4), metrics=['accuracy'])
model.summary()
```

옆 코드와 같이
6층 인공지능망 모델을 생성

다음은 **인공신경망 네트워크**를 구성하여 이미지 분류를 위한 모델을 만든다.

3층 인공신경망 모델의 **입력층**, **은닉층**, **출력층**으로 **신호 전달하는 과정**



3층 인공신경망 모델

$$(2, 3) \quad \begin{pmatrix} w_{11}^{(1)} & w_{21}^{(1)} & w_{31}^{(1)} \\ w_{12}^{(1)} & w_{22}^{(1)} & w_{32}^{(1)} \end{pmatrix} \quad (b_1^{(1)}, b_2^{(1)}, b_3^{(1)})$$

$$(3, 2) \quad \begin{pmatrix} w_{11}^{(2)} & w_{21}^{(2)} \\ w_{12}^{(2)} & w_{22}^{(2)} \\ w_{13}^{(2)} & w_{23}^{(2)} \end{pmatrix} \quad (b_1^{(2)}, b_2^{(2)})$$

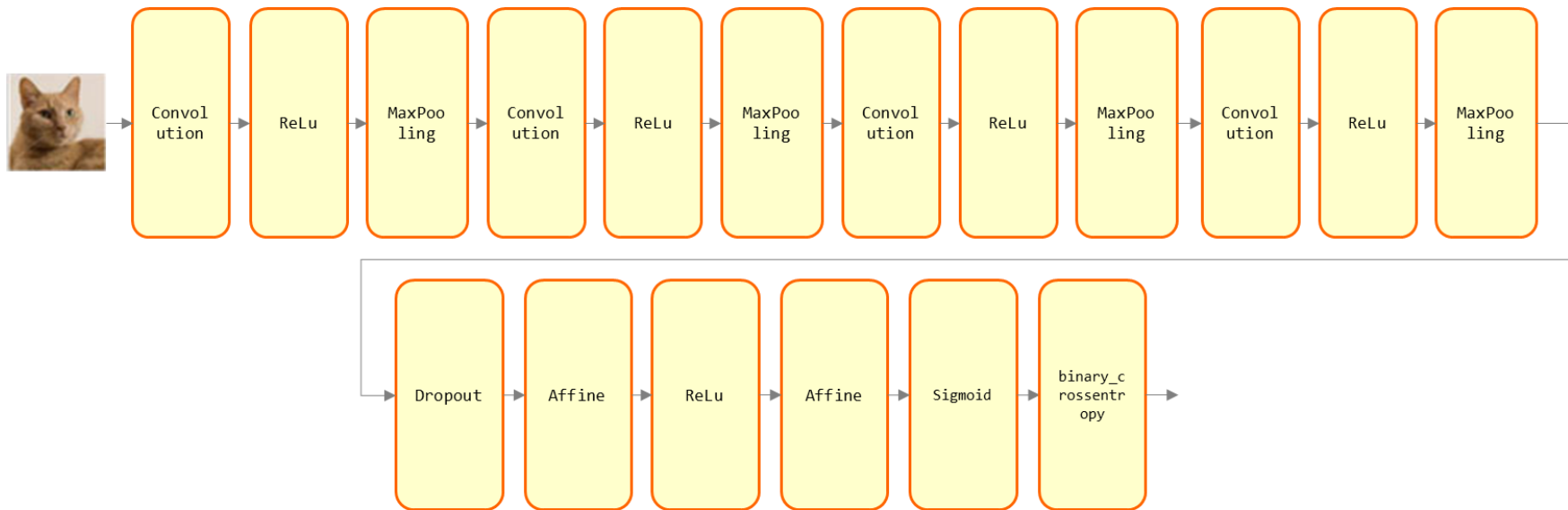
$$(2, 2) \quad \begin{pmatrix} w_{11}^{(3)} & w_{21}^{(3)} \\ w_{12}^{(3)} & w_{22}^{(3)} \end{pmatrix} \quad (b_1^{(3)}, b_2^{(3)})$$

가중치

편향

다음은 **인공신경망 네트워크**를 구성하여 이미지 분류를 위한 모델을 만든다.

6층 인공신경망 모델의 계층구조로 이루어진 네트워크 구조



인공신경망 네트워크 구성

다음은 **인공신경망 네트워크**를 구성하여 이미지 분류를 위한 모델을 만든다.

인공신경망 네트워크 구성 결과

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 148, 148, 32)	896
max_pooling2d (MaxPooling2D)	(None, 74, 74, 32)	0
conv2d_1 (Conv2D)	(None, 72, 72, 64)	18496
max_pooling2d_1 (MaxPooling2D)	(None, 36, 36, 64)	0
conv2d_2 (Conv2D)	(None, 34, 34, 128)	73856
max_pooling2d_2 (MaxPooling2D)	(None, 17, 17, 128)	0
conv2d_3 (Conv2D)	(None, 15, 15, 128)	147584
max_pooling2d_3 (MaxPooling2D)	(None, 7, 7, 128)	0
flatten (Flatten)	(None, 6272)	0
dropout (Dropout)	(None, 6272)	0
dense (Dense)	(None, 512)	3211776
dense_1 (Dense)	(None, 1)	513

Total params: 3,453,121

Trainable params: 3,453,121

Non-trainable params: 0

/usr/local/lib/python3.7/dist-packages/keras/optimizer_v2/rmsprop.py:130: UserWarning: The `lr` argument is deprecated, use `learning\_rate` instead.
super(RMSprop, self).__init__(name, **kwargs)

다음은 모델 학습 시 과적합(Overfitting) 방지를 위해 학습을 조기에 종료시키는 콜백함수인 EarlyStopping() 클래스의 옵션을 설정한다.

```
# 모델 학습시 정확도를 모니터링하여 조기에 학습을 종료시키는 인스턴스 정의

# Tensorflow, 케라스 콜백함수 EarlyStopping() :
# - 모델을 더 이상 학습을 못할 경우(loss, metric등의 개선이 없을 경우), 학습 도중 미리 학습을 종료시키는 콜백함수이다.

monitor_val_acc = EarlyStopping(monitor='val_accuracy', patience=5)
```

- ➡ EarlyStopping 클래스의 monitor 인자 값에 'val_accuracy'를 입력하면, val_accuracy가 더 이상 증가하지 않을 경우 EarlyStopping을 적용함
- ➡ EarlyStopping 클래스의 patience 인자는 학습이 진행됨에도 더 이상 monitor되는 값의 개선이 없을 경우, 최적의 monitor 값을 기준으로 몇 번의 에폭(epoch)을 진행할 지정하는 값임

☐ 예 patience=3 인자 값을 입력하면 10 epoch에 정확도가 99%였을 때,
만약 11 epoch에 정확도 98%, 12epoch에 정확도 98.1%, 13epoch에 98.3%라면
더 이상 학습을 진행하지 않고 종료시킴

다음은 정의된 **인공신경망 모델**을 **학습**시켜 보자.

```

1  # model training(모델을 학습시킵니다)
2  history = model.fit_generator(generator=train_generator,
3                               steps_per_epoch=2000//batch_size,
4                               epochs=no_of_epochs,
5                               validation_data=test_generator,
6                               validation_steps=800//batch_size,
7                               callbacks=[monitor_val_acc])
8  # 정확도를 모니터링하다가 증가되지 않고 떨어지거나 변화가 거의 없다면 patience 만큼 기다리다 학습을 중단함
    
```

- 위 코드에서 모델 학습 중 `callbacks = [monitor_val_acc]`에 의해서 **정확도를 모니터링**하다가 **증가되지 않고 떨어지거나 변화가 거의 없다면** `patience=5`(위 예제의 경우) 만큼 기다리다 학습을 조기에 중단시킴

다음은 정의된 인공신경망 모델을 학습시켜 보자.

```
Epoch 1/50
/usr/local/lib/python3.7/dist-packages/ipynb_launcher.py:7: UserWarning: `Model.fit_generator` is deprecated and will be removed in a future version. Please use `Model.fit`, which supports generators.
  import sys
200/200 [=====] - 22s 109ms/step - loss: 0.5071 - accuracy: 0.7470 - val_loss: 0.5029 - val_accuracy: 0.7450
Epoch 2/50
200/200 [=====] - 22s 109ms/step - loss: 0.4971 - accuracy: 0.7585 - val_loss: 0.5182 - val_accuracy: 0.7525
Epoch 3/50
200/200 [=====] - 22s 110ms/step - loss: 0.4964 - accuracy: 0.7705 - val_loss: 0.5009 - val_accuracy: 0.7563
Epoch 4/50
200/200 [=====] - 22s 111ms/step - loss: 0.5080 - accuracy: 0.7500 - val_loss: 0.4830 - val_accuracy: 0.7550
Epoch 5/50
200/200 [=====] - 22s 109ms/step - loss: 0.5013 - accuracy: 0.7575 - val_loss: 0.5028 - val_accuracy: 0.7425
Epoch 6/50
200/200 [=====] - 22s 109ms/step - loss: 0.4950 - accuracy: 0.7595 - val_loss: 0.4577 - val_accuracy: 0.7812
Epoch 7/50
200/200 [=====] - 22s 109ms/step - loss: 0.5029 - accuracy: 0.7500 - val_loss: 0.5090 - val_accuracy: 0.7487
Epoch 8/50
200/200 [=====] - 23s 114ms/step - loss: 0.5017 - accuracy: 0.7555 - val_loss: 0.4932 - val_accuracy: 0.7638
Epoch 9/50
200/200 [=====] - 22s 111ms/step - loss: 0.4806 - accuracy: 0.7700 - val_loss: 0.4855 - val_accuracy: 0.7775
Epoch 10/50
200/200 [=====] - 22s 111ms/step - loss: 0.4943 - accuracy: 0.7605 - val_loss: 0.4782 - val_accuracy: 0.7900
Epoch 11/50
200/200 [=====] - 22s 109ms/step - loss: 0.4808 - accuracy: 0.7655 - val_loss: 0.5581 - val_accuracy: 0.7550
Epoch 12/50
200/200 [=====] - 22s 108ms/step - loss: 0.4762 - accuracy: 0.7735 - val_loss: 0.5405 - val_accuracy: 0.7400
Epoch 13/50
200/200 [=====] - 23s 115ms/step - loss: 0.4875 - accuracy: 0.7715 - val_loss: 0.5178 - val_accuracy: 0.7425
Epoch 14/50
200/200 [=====] - 22s 108ms/step - loss: 0.4814 - accuracy: 0.7740 - val_loss: 0.5036 - val_accuracy: 0.7738
Epoch 15/50
200/200 [=====] - 22s 109ms/step - loss: 0.4805 - accuracy: 0.7735 - val_loss: 0.4855 - val_accuracy: 0.7788
```

실행 결과 10 epoch에서 정확도가 79%이고, 11~15epoch 학습을 진행하였지만
정확도가 개선되지 않아 학습이 완료된 것을 볼 수 있음

다음은 테스트 데이터 셋으로 학습된 인공지능망 모델을 평가해 보자.

```
# model evaluate(모델을 평가해 봅니다)
score = model.evaluate(test_generator, steps=800//batch_size, verbose=0)
print('Test loss:', score[0])
print('Test accuracy:', score[1]*100)
```

Test loss: 0.4854712188243866

Test accuracy: 77.87500023841858

실행 결과 테스트 데이터 셋에 대한 정확도가 77.87% 나온 것을 볼 수 있음

다음은 학습된 인공지능 모델에 대해 훈련 데이터 셋과 검증용 데이터 셋의 손실값(Loss), 정확도(Accuracy)의 추이를 그래프로 표현해 보자.



```
plt.figure(figsize=(16,5))
```

```
# 만들어진 모델에 대해 train dataset과 validation dataset의 loss 를 그래프로 표현해 봅니다.
```

```
plt.subplot(1, 2, 1)
```

```
plt.plot(history.history['loss'])
```

```
plt.plot(history.history['val_loss'])
```

```
plt.title('model loss')
```

```
plt.ylabel('loss')
```

```
plt.xlabel('epoch')
```

```
plt.legend(['train', 'validation'], loc='upper left')
```

```
# 만들어진 모델에 대해 train dataset과 validation dataset의 accuracy 를 그래프로 표현해 봅니다.
```

```
plt.subplot(1, 2, 2)
```

```
plt.plot(history.history['accuracy'])
```

```
plt.plot(history.history['val_accuracy'])
```

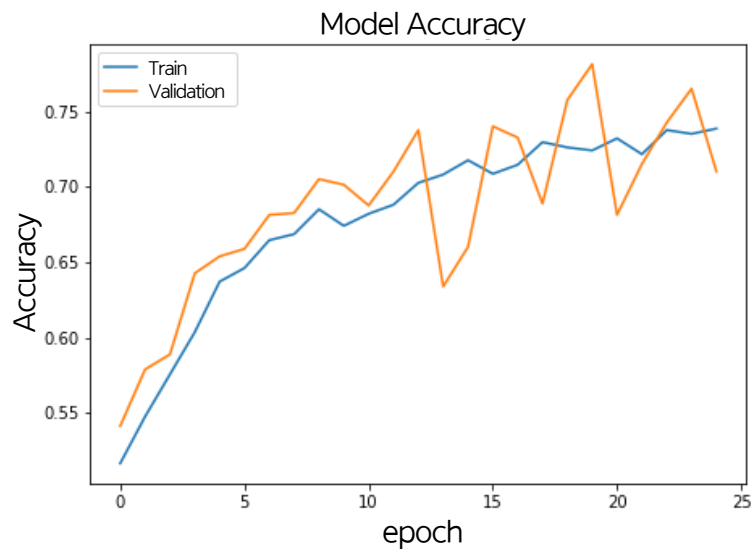
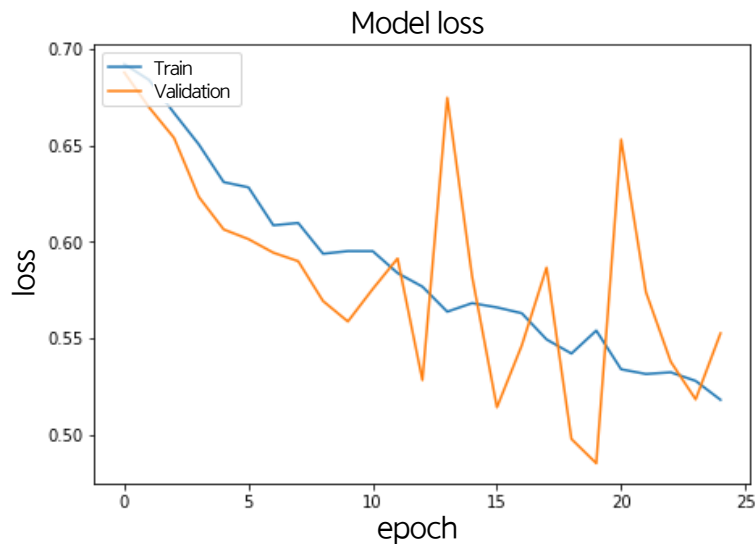
```
plt.title('model accuracy')
```

```
plt.ylabel('accuracy')
```

```
plt.xlabel('epoch')
```

```
plt.legend(['train', 'validation'], loc='upper left')
```

다음은 학습된 인공신경망 모델에 대해 **훈련 데이터 셋**과 **검증용 데이터 셋**의 **손실값(Loss)**, **정확도(Accuracy)**의 추이를 그래프로 표현해 보자.



- 훈련 데이터 셋의 경우 손실값이 점차 줄어 학습이 잘 진행되는 것으로 보임
- 훈련 데이터 셋의 경우 정확도가 점차 높아지고 있어 학습이 잘 진행되는 것으로 보임

다음은 학습한 **인공신경망 모델**을 **H5 파일 형식**으로 **저장**한다.

```
# 전체 모델을 H5 파일로 저장합니다.  
model.save('Cats_vs_Dogs_model.h5')  
print("Saved model to disk")
```

Saved model to disk

실행 결과 **Cats_vs_Dogs_model.h5** 파일 이름으로 VM에 **저장**된 것을 알 수 있음



학습된 모델에 `pred_generator` 객체가 만들어낸 이미지로 분류 예측을 수행해 보자.

여기서는 16장 이미지를 분류 예측하고 그 결과를 `predictions` 객체에 저장함



```
# pred_generator 가 만들어낸 이미지를 우리가 만든 모델을 통해 분류 예측해 봅니다.  
predictions = model.predict(pred_generator, steps=16, verbose=1)
```



```
16/16 [=====] - 0s 6ms/step
```

학습된 모델에 `pred_generator` 객체가 만들어낸 이미지로 분류 예측을 수행해 보자.

- 다음은 `predictions` 객체의 형상과 예측결과를 확인한 것임
- 예측결과 값의 분포는 0 ~ 1사이의 실수인 것을 알 수 있음
- 즉 0 ~ 0.5 이하의 경우 'Cat', 0.5 보다 큰 값은 'Dog'로 분류됨

```
1 # predictions shape 확인
2 predictions.shape
```

```
(16, 1)
```

```
1 # 예측결과 확인
2 predictions
```

```
array([[0.1696523 ],
       [0.3645607 ],
       [0.07953901],
       [0.53780824],
       [0.6138283 ],
       [0.06508067],
       [0.02131473],
       [0.9830064 ],
       [0.50652534],
       [0.7645795 ],
       [0.98972636],
       [0.38176018],
       [0.34120196],
       [0.4365621 ],
       [0.13959974],
       [0.99547064]], dtype=float32)
```

0 ~ 0.5 이하 : Cat
0.5 보다 큰 값 : Dog



다음은 `pred_generator` 객체가 만들어 내는 이미지의 클래스 인덱스를 확인해 보자.

```
# pred_generator 가 만들어 내는 이미지의 클래스 인덱스가 무엇인지 확인해 본다.  
print(pred_generator.class_indices)
```

```
{'cat': 0, 'dog': 1}
```

- 실행 결과 고양이는 0, 개는 1임을 알 수 있음
- 위 코드에서 `class_indices` 객체 변수의 자료구조는 딕셔너리임

다음은 학습된 모델이 예측한 16장 이미지의
예측결과(predictions 객체에 저장된 결과)를 시각화한 것이다.

```
1 # Predict Result(pred_generator 가 만들어 낸 이미지를 우리가 만든 모델을 통해 예측한 결과를 보여줍니다.)
2 f,ax = plt.subplots(4,4,figsize=(20,20))
3
4 for i in range(len(predictions)):
5     image, label = pred_generator.next() # pred_generator 가 만들어 낸 이미지를 하나씩 꺼내온다
6     real_class = 'cat' if int(label) == 0 else 'dog' # pred_generator 가 만들어 낸 이미지의 label name 을 가져옴
7     ax[i//4,i%4].imshow(image[0]) # pred_generator 가 만들어 낸 이미지를 보여줌
8     ax[i//4,i%4].axis('off')
9
10    pred_class = 'dog' if predictions[i] > 0.5 else 'cat' # pred_generator 가 만들어 낸 이미지를 모델을 통해 예측한 결과 값의 label name
11    pred_prob = predictions[i] # pred_generator 가 만들어 낸 이미지를 모델을 통해 예측한 결과(확률값)
12    ax[i//4,i%4].set_title('REAL:' + real_class + '\nPREDICT:' + pred_class + '(' + str(pred_prob) + ')') # title 에 실제값과 모델에 의하
13
14 f.suptitle('Predict Result')
15
16 plt.show()
```

위 코드에서 predictions 객체의 원소 값이 0.5 이하이면 'Cat',
0.5보다 크면 'Dog'로 예측한 것으로 판단하고 있음

다음은 학습된 모델이 예측한 16장 이미지의
예측결과(predictions 객체에 저장된 결과)를 시각화한 것이다.

예측결과를 보면 모델은 정확도 50%로 분류한 것을 알 수 있음

정답 : 8

오답 : 8

